Programación Concurrente y Paralela

Práctica 8: Cómputo Paralelo

2do Cuatrimestre 2026

Ejercicio 1. Para este ejercicio, supongamos que podemos estimar T_P usando la cota de un scheduler greedy, es decir, $T_P \approx \frac{T_1}{P} + T_\infty$. Romina está buscando optimizar un programa paralelo Π y logró desarrollar una versión Π' que reduce la cantidad de pasos totales del programa de 2048 a 1024. Haciendo pruebas en la computadora de su laboratorio con 32 procesadores, observa que la cantidad de pasos que requiere ejecutar Π' es menor a la que requiere ejecutar Π, pero antes de solicitar tiempo de cómputo en una computadora de 512 procesadores se hace la pregunta: ¿Qué condición debería cumplirse, suponiendo la estimación de T_P con la que contamos, para que a pesar de tomar menos pasos Π' con 32 procesadores, tome *más* pasos si se corriera con 512 procesadores?[1]

Ejercicio 2. Consideremos la cota superior para el tiempo de cómputo con un scheduler greedy en P procesadores T_P^*. Muestre que la cota $T_P^* \leq \frac{T_1}{P} + T_\infty$ puede de bajarse a $T_P^* \leq \frac{T_1 - T_\infty}{P} + T_\infty$. (Idea: usando la misma partición en c pasos completos y k pasos incompletos, y argumente una nueva cota más ajustada para c)(Nota: se asume $P > 1$)

Ejercicio 3. Felipe estuvo midiendo su programa determinístico multiprocesador Π en una plataforma que le garantiza un scheduler greedy. Encontró que para Π, $T_4 = 80$ pasos de ejecución y $T_{64} = 10$ pasos de ejecución. Usando la cota del ejercicio 2 más las leyes de *work* y *span*, determine el máximo de pasos de ejecución que tomaría el programa Π si se ejecutara en un equipo con 10 procesadores.

Ejercicio 4. Escriba la función `vectorSum` que dado un vector `A` de longitud n potencia de 2, use el estilo *fork-join* para realizar la suma de todos los elementos de `A`. Analice el *work*, *span* y paralelismo de la solución.

Ejercicio 5. Escriba la función `power` que tome un entero `X` junto con un exponente `c` y obtenga el valor de X^c usando la técnica de potenciación por cuadrados (Es decir, calculando $X^{\lfloor c/2 \rfloor}$ y luego elevando el resultado al cuadrado) usando el estilo *fork-join* (asumiendo multiplicaciones de costo $\Theta(1)$). ¿Qué forma tiene el DAG de cómputo en este caso? Calcule el *work*, *span* y paralelismo de la solución.

Ejercicio 6. Escriba la función `transponer` que dada una matriz cuadrada `A` de lado n potencia de 2 calcule su *transpuesta*, es decir la matriz `A_T` tal que `A[i][j]=A_T[j][i]`, usando el estilo *fork-join*. (Idea: parta la matriz en cuatro cuadrantes y observe a qué cuadrante de la transpuesta va cada uno.) Analice el *work*, *span* y paralelismo del mismo.

Ejercicio 7. Considere el siguiente pseudocódigo estilo *fork-join* para hacer *mergesort*:

```
void mergeSort(int[] A, int lo, int hi) {
```

[1] Este ejercicio está inspirado en uno del libro de Herilhy, que a su vez surge de una historia real sobre comparación de programas de Ajedrez que está contada en el libro de Cormen.

```
        if (hi > lo) {
            int mid = (lo + hi) / 2;
            low = new mergeSort(A, lo, mid);
            low.fork();
            compute(new mergeSort(A, mid + 1, hi));
            low.join();
            merge(A, lo, mid, hi);
        }
    }
```

Asumiendo que `merge` no tiene paralelismo interno y toma $\Theta(n)$ pasos sobre un segmento de tamaño n, determine el *work*, el *span* y el paralelismo del algoritmo. Escriba las respuestas tanto como recurrencias como en notación $\Theta(f(n))$, para alguna función f del tamaño de la entrada n.

Ejercicio 8. El `merge` secuencial es el cuello de botella del *mergesort* paralelo del ejercicio 7. En este ejercicio lo paralelizamos, manteniendo el mismo esquema *fork-join*. Para simplificar, vamos a suponer que todos los elementos son distintos.

Dados dos segmentos ordenados `T[p1..r1]` y `T[p2..r2]` de un array `T` con n elementos en total, la estrategia es partir la unión alrededor de su *mediana* x. Suponiendo que contamos con las siguientes funciones ya implementadas:

- `median(T, p1, r1, p2, r2)`: devuelve un par `(p, q)`, donde `p` $\in$ {`p1`, `p2`} es el comienzo del segmento donde está la mediana, y `q` es el índice (en `T`) del valor de la mediana de los valores que constituyen ambos segmentos. Este cálculo se hace secuencialmente[2] en $\Theta(\log n)$.
- `binarySearch(x, T, p, r)`: devuelve la menor posición $q \in [p, r+1]$ tal que $T[q] > x$ (o $r+1$ si no existe), calculada secuencialmente en $\Theta(\log n)$.

El pseudocódigo de `merge(T, p1, r1, p2, r2, A, p3)` se puede escribir como:

```
void merge(int[] T, int p1, int r1,
           int p2, int r2, int[] A, int p3) {
  int n1 = r1 - p1 + 1, n2 = r2 - p2 + 1;
  if (n1 + n2 == 0) {
    return;  //No hay valores en los segmentos a analizar
  }
  int p, q = median(T, p1, r1, p2, r2); //mediana: T[q]
  int val = T[q];
  int base_op, fin_op = (p == p1) ? (p2, r2) : (p1, r1);
  int q_op = binarySearch(val, T, base_op, fin_op);
  int pos_en_A = p3 + (q - p) + (q_op - base_op);
  A[pos_en_A] = val;
  //izq: [p..q-1] y [base_op..q_op-1]; der: el resto
  //uso p para identificar los segmentos
```

[2]Un ejercicio interesante (aunque capaz más de Algoritmos que de esta materia) es escribir esa función

```
    int low_r1, low_r2 =
        (p == p1) ? (q-1, q_op-1) : (q_op-1, q-1);
    int hi_p1, hi_p2 =
        (p == p1) ? (q+1, q_op) : (q_op, q+1);
    low = new merge(T, p1, low_r1, p2, low_r2, A, p3);
    low.fork();
    compute(new merge(T, hi_p1, r1,
                      hi_p2, r2, A, pos_en_A+1));
    low.join();
}
```

a) Justifique por qué las dos fusiones recursivas pueden ejecutarse en paralelo sin condiciones de carrera, y por qué el resultado es correcto.

b) Analice el *work*, el *span* y el paralelismo de `merge` como recurrencias y en notación Θ.

c) Vuelva a analizar el *work*, el *span* y el paralelismo del `mergesort` si se utiliza este *merge* en vez de la versión lineal.